

Phase 6C · Stand up the app on AWS

OWNER · ~2 HRS

Phases 1 to 4 built the EMR's half of the boundary. This phase builds the app's half: a **second, independent stack beside OpenEMR**, in the same BAA account. Nothing here changes the EMR — `gfc-emr-web`, `gfc-emr-db`, `gfc-openemr-db`, `gfc-openemr` and the `emr` DNS record all stay exactly as they are.

The steps mirror Phases 1 to 4 almost exactly. Different names, and Postgres instead of MySQL — the app needs Postgres, so sharing OpenEMR's database was never an option.

Why the app does not run on the OpenEMR box

That instance is a `t3.small` already carrying OpenEMR and its MySQL connections. Adding a Node process makes both slower, and worse, ties their fate together — a restart for one takes the other down. Bethel should not lose charting because the app needed a bounce. Separate instances, separate failure domains.

What this runs on, and what it adds

ITEM	VALUE
Application	GFC Care Platform, Node 22 in Docker, built from the <code>Dockerfile</code> in the repo
Host	EC2 <code>t3.small</code> , Ubuntu 24.04, 20 GB encrypted <code>gp3</code> , instance name <code>gfc-app</code> , Elastic IP
Database	RDS PostgreSQL 16 , <code>db.t4g.micro</code> , encrypted, 20 GB <code>gp3</code> autoscaling to 100 GB, 30-day automated backups, identifier <code>gfc-app-db</code>
Address	<code>https://app.godwinsfamilycarellc.com</code> (Let's Encrypt, issued by Caddy)
TLS	Caddy reverse proxy in the same compose file. An ALB does the same job for about \$16/month; at this scale Caddy is the better trade
Region	<code>us-east-1</code> , same as the EMR. Everything in one region, always
Added run cost	About \$30 to \$35 per month. Raise the budget alarm from \$80 to about \$140 to cover both stacks
Documents	Unchanged — consents and signed PDFs stay in HIPAA Google Drive under the Workspace BAA

6C.1 · Security groups

10 MIN

- `gfc-app-web`, "GFC app web server". Inbound: HTTPS 443 from Anywhere-IPv4; HTTP 80 from Anywhere-IPv4 (Let's Encrypt's HTTP-01 challenge needs it); SSH 22 from My IP; SSH 22 from `18.206.107.24/29` so the browser terminal can reach the box. Outbound default.
- `gfc-app-db`, "GFC app database". Inbound: PostgreSQL 5432 with source `gfc-app-web`. Nothing else. The database is never exposed to the internet.

6C.2 · The database (RDS Postgres)

20 MIN + 10 MIN WAIT

- 1 RDS → Create database → Standard create → **PostgreSQL** (latest 16.x) → template Production → Single DB instance.
- 2 Identifier gfc-app-db. Master username appadmin. Master password generated into the password manager as APP-DB-MASTER.
- 3 db.t4g.micro. Storage 20 GB gp3, autoscaling on, max 100 GB.
- 4 Connectivity: do not connect to an EC2 resource; default VPC; public access **No**; security group gfc-app-db only.
- 5 Additional configuration: **leave "Initial database name" blank**. The app creates its own schema on first boot; pre-creating it causes the first migration to fail. Backup retention 30 days, **backup window off clinic hours**. Encryption enabled. Deletion protection on.
- 6 When *Available*, copy the endpoint to the worksheet. It is the host portion of DATABASE_URL.

6C.3 · The app server and a permanent address

20 MIN

- 1 EC2 → Launch instance. Name gfc-app. Ubuntu Server 24.04 LTS x86. t3.small. Key pair gfc-emr-key (reuse; break-glass only).
- 2 Network settings → Edit: auto-assign public IP Enable; Firewall → select existing → gfc-app-web.
- 3 Storage 20 GB gp3 → Advanced → **Encrypted: Yes** (default KMS key). Launch.
- 4 **Permanent address:** EC2 → Elastic IPs → Allocate → Associate → instance gfc-app. Copy it to the worksheet. Without this the address changes on every restart and the domain breaks.

6C.4 · Point the domain at it

10 MIN + PROPAGATION

- 1 Log in wherever godwinsfamilycarellc.com's DNS is managed — the same place the emr record was added in Phase 4.
- 2 Add a record: Type **A** · Host/Name app · Value: the Elastic IP · TTL default (lower it to 300 the day before cutover).
- 3 Verify at dnschecker.org before continuing: look up app.godwinsfamilycarellc.com (type A). When it returns the Elastic IP from most locations, proceed.

Do not start 6C.5 until DNS resolves

Caddy requests the HTTPS certificate on first start and the certificate authority verifies the domain by visiting it. If DNS is not pointing at the server yet, issuance fails and you repeat the step.

6C.5 · Install and run the app

30 MIN

Everything below is typed in the browser terminal: **EC2** → **Instances** → **gfc-app** → **Connect** → **EC2 Instance Connect**. Same as Phase 5. Nothing touches your Mac.

Prepare the server. Paste each block, press Enter, wait for the prompt.

```
sudo apt-get update && sudo apt-get -y upgrade
sudo apt-get -y install docker.io docker-compose-v2 git
sudo systemctl enable --now docker
sudo usermod -aG docker ubuntu && newgrp docker
```

Get the code. The repository is private, so use a deploy key or a fine-grained token with Contents: read.

```
sudo mkdir -p /opt/gfc && sudo chown ubuntu:ubuntu /opt/gfc
cd /opt/gfc
git clone https://github.com/bume1/Godwins-Family-Care.git app
cd app
```

Write the Caddy config. Two lines. Caddy obtains and renews the certificate on its own.

```
cat > /opt/gfc/app/Caddyfile <<'EOF'
app.godwinsfamilycarellc.com {
    reverse_proxy app:3000
}
EOF
```

Write the compose file.

```
cat > /opt/gfc/app/compose.yaml <<'EOF'
services:
  app:
    build: .
    restart: always
    env_file: /opt/gfc/app.env
    expose: ["3000"]

  caddy:
    image: caddy:2-alpine
    restart: always
    ports: ["80:80", "443:443"]
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    depends_on: [app]

volumes:
  caddy_data:
  caddy_config:
EOF
```

Generate the two new secrets. Everything else comes from the worksheet.

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))" # JWT_SECRET
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))" # EMR_TOKEN_ENCRYPTION_KEY
```

Write the environment file. Kept outside the repo directory so a git pull can never touch it, root-owned and unreadable by anyone else.

```
umask 077
cat > /opt/gfc/app.env <<'EOF'
NODE_ENV=production
DATA_STORE=postgres
DATABASE_URL=postgres://appadmin:<APP-DB-MASTER>@<RDS-ENDPOINT>:5432/gfc?sslmode=verify-full
JWT_SECRET=<generated above>
EMR_TOKEN_ENCRYPTION_KEY=<generated above>
OPENEMR_BASE_URL=https://emr.godwinsfamilycarellc.com
OPENEMR_SITE=default
OPENEMR_CLIENT_ID=<v4 client id>
OPENEMR_CLIENT_SECRET=<v4 client secret>
OPENEMR_REDIRECT_URI=https://app.godwinsfamilycarellc.com/oauth/callback
EOF
sudo chown root:root /opt/gfc/app.env && sudo chmod 600 /opt/gfc/app.env
```

Secrets: interim versus intended

Part VI control C-9 specifies AWS Secrets Manager. A root-owned 600 file on an encrypted volume inside the boundary is a defensible interim and is what the commands above produce. Secrets Manager with an instance IAM role is the stronger posture and the intended end state. If you go live on the file, record it as a known deviation with a review date.

Launch.

```
cd /opt/gfc/app
docker compose up -d --build
docker compose logs -f
```

The first build takes a few minutes. Watch for the boot line naming the adapter: `Data store: postgres` (production, inside the BAA boundary). Press Ctrl+C to leave the log view; the server keeps running.

If it refuses to boot, read the message — that is the guard working

The app deliberately will not start in production unless `DATA_STORE=postgres`, `DATABASE_URL`, `JWT_SECRET` and `EMR_TOKEN_ENCRYPTION_KEY` are all set, and it refuses `MFA_ENFORCE=false` outright. Every refusal names its own reason. A refusal is the control doing its job, not a failure of the install.

6C.6 · Verify, back up, and hand off to Session 5

15 MIN

- 1 From your own browser: `https://app.godwinsfamilycarellc.com/healthz`. Expected: `{"ok":true,"store":"postgres","production":true}`, a valid padlock, no warning. This is the line the Session 5 runbook checks as preflight P1.
- 2 **Snapshots.** EC2 → Elastic Block Store → Lifecycle Manager → create an EBS snapshot policy targeting the `gfc-app` volume, daily, 14-day retention. Same pattern as Phase 6 for the EMR box.
- 3 **Cost alarm.** Billing → Budgets → raise the monthly budget from \$80 to about \$140 to cover both stacks.
- 4 **Logs.** CloudWatch → create a log group for the app with a retention period set. Application logs are PII-scrubbed by Session 5, but keep them inside the account regardless.
- 5 **Monthly patch routine** — add to the Part VI calendar alongside the EMR routine: `cd /opt/gfc/app && git pull && docker compose up -d --build`.

What this phase does and does not finish

6C satisfies preflight **P1** (in-boundary host with TLS) and **P2** (encrypted RDS) of the Session 5 cutover runbook. It does **not** move any data — the store is empty and the app is not yet serving anyone. The migration from Replit KV, the DNS switch for users, the Replit decommission and the go/no-go all belong to the runbook's §4 sequence, which starts the moment /healthz answers correctly.

Adding features after the move. Development does not change. Claude Code edits and merges as before; the only difference is that deployment becomes one deliberate command on the box instead of an automatic Replit redeploy. Build features after the transition rather than before — anything built on Replit has to be retested here anyway, and cannot be exercised against real data until it lives inside the boundary.